Practice Questions

Claude Certified Architect - Foundations | no answer keys

No answers in this booklet. That is deliberate. Write your answers on paper, then mark them on screen against the answer files in `prep/practice/`.

The questions use exam-level English, the same long formal style as the real exam. Reading that style quickly is part of the test, so they are not simplified.

Take each drill only **after** you have read that domain's notes. Taking one early wastes the questions.

Timing: Set 1 is 40 minutes. Each 15-question drill is 30 minutes. Watch the clock - the real exam gives you about 2 minutes per question.

Contents

Practice Set 1 - Diagnostic (20 items)

Blueprint-weighted: D1 x5 - D2 x4 - D3 x4 - D4 x4 - D5 x3. Target time: **40 minutes** (2 min/item). Select one response unless stated otherwise.

Write your answers down before checking `set-01-answers.md`. Score by domain - the point of this set is to find your weak domain, not to feel good.

Scenario: Customer Support Resolution Agent

Q1. Your support agent calls `lookup_order` with a customer-supplied order number, and the MCP tool times out because the orders service is briefly unavailable. The tool currently returns `{"isError": true, "message": "Operation failed"}`. The agent responds by telling the customer their order does not exist. What change most directly prevents this failure mode?

A. Increase the tool's timeout so transient outages resolve before the call returns. B. Return structured error metadata including `errorCategory`, an `isRetryable` boolean, and a human-readable description. C. Add a system prompt instruction telling the agent never to claim an order doesn't exist unless it is certain. D. Have the tool return an empty result set so the agent treats it as a no-match case and asks for clarification.

Q2. Your agent has an 88% first-contact resolution rate but audit review finds that in several cases it processed refunds for a customer whose identity had been verified in a *different* conversation earlier that day. Refunds must never be issued without verification in the current session. What is the correct fix?

A. Add a few-shot example showing the agent re-verifying identity at the start of every session. B. Add a `session_verified` boolean to the system prompt context and instruct the agent to check it. C. Implement a programmatic prerequisite that blocks `process_refund` until `get_customer` has returned a verified customer ID in the current session. D. Reduce the agent's tool set so `process_refund` is only enabled after a routing classifier detects a refund intent.

Q3. A customer writes: "This is the third time I've contacted you about this broken blender. Just give me a human, I'm done explaining this." Your agent has full capability to process the replacement. What should it do?

A. Acknowledge the frustration, offer to process the replacement immediately, and escalate only if the customer reiterates their preference. B. Escalate to a human immediately, compiling a structured handoff summary. C. Process the replacement without escalating, since resolving it is the fastest path to satisfaction. D. Run a sentiment check and escalate if negative sentiment exceeds the configured threshold.

Q4. Your `lookup_order` MCP tool returns 43 fields per order. The agent only ever needs order status, item names, purchase date, refund eligibility, and total. Over long multi-issue conversations, context fills with order payloads and the agent starts giving inconsistent answers about earlier issues. Which two changes address this? **(Select two.)**

A. Trim the tool's output to the relevant fields before the results enter the conversation context. B. Extract structured issue data (order IDs, amounts, statuses) into a separate persistent context layer. C. Instruct the agent in the system prompt to ignore irrelevant fields in tool results. D. Switch to a model with a larger context window so the full payloads fit comfortably.

Scenario: Multi-Agent Research System

Q5. Your coordinator delegates to a document-analysis subagent, which returns excellent summaries. But the final reports attribute claims to the wrong sources, and some claims have no source at all. The document-analysis subagent's output is a well-written prose summary of each document. What is the most likely root cause?

A. The synthesis subagent lacks instructions to preserve attribution when merging findings. B. The document-analysis subagent returns prose without structured claim-source mappings, so attribution is lost at the summarization step. C. The coordinator is not passing source URLs to the synthesis subagent. D. The report-generation subagent is reformatting citations incorrectly.

Q6. Your coordinator agent is configured with `allowedTools: ["Read", "Grep", "WebSearch"]` and a system prompt instructing it to delegate document analysis to a specialized subagent. In testing, it never delegates - it analyzes documents itself. What is the cause?

A. The subagent's AgentDefinition description is too vague for the coordinator to select it. B. `allowedTools` does not include `"Task"`, so the coordinator has no mechanism to spawn subagents. C. The coordinator's system prompt needs stronger language emphasizing that delegation is mandatory. D. Subagents must be registered in `.mcp.json` before a coordinator can invoke them.

Q7. Two credible sources in your research report state different figures for the same market size: one says $4.2B (published 2024, surveying North America) and one says $6.8B (published 2026, global). Your synthesis agent currently picks the more recent figure. What should it do instead?

A. Average the two figures and note the range in a footnote. B. Preserve both values with source attribution and methodological context, and structure the report to distinguish well-established from contested findings. C. Route the conflict to the coordinator, which selects the figure from the more authoritative source. D. Discard both and re-delegate to the web-search subagent for a third, tie-breaking source.

Q8. Your coordinator delegates four research subtasks. It currently emits one Task tool call, waits for the result, then emits the next. Total latency is 4x a single subagent run. What change enables parallel execution?

A. Configure each subagent's AgentDefinition with a `parallel: true` flag. B. Emit all four Task tool calls in a single coordinator response. C. Use `fork_session` to create four branches from the coordinator's baseline analysis. D. Increase the coordinator's `allowedTools` limit so it can hold more concurrent tool calls.

Scenario: Code Generation with Claude Code

Q9. Your team's monorepo has three packages, each with distinct API conventions. You want each package's maintainers to control which shared standards documents apply to their package, without duplicating those documents. What is the most maintainable approach?

A. Place a CLAUDE.md in each package directory that uses `@import` to reference the relevant standards files. B. Create `.claude/rules/` files with `paths:` globs for each package directory. C. Concatenate all standards into the root CLAUDE.md under per-package headers. D. Create a skill per package in `.claude/skills/` containing that package's standards.

Q10. A developer reports that Claude Code applies your team's commit-message conventions inconsistently - sometimes correct, sometimes not, across sessions on the same repo. What should they run first to diagnose it?

A. `/compact` B. `/memory` C. `claude --resume` D. `/doctor`

Q11. You've built a skill that performs a full-codebase dependency analysis. It produces several thousand lines of output. Developers report that after invoking it, Claude Code seems to lose track of the task they were working on. Which SKILL.md frontmatter option addresses this?

A. `allowed-tools: [Read, Grep, Glob]` B. `argument-hint: <package-name>` C. `context: fork` D. `paths: ["**/*"]`

Q12. You are assigned to add a null check to a single utility function after a stack trace identified the exact line. Which approach is appropriate?

A. Plan mode, to explore whether other call sites have the same problem before changing anything. B. Direct execution. C. Plan mode, because any change to a shared utility has architectural implications. D. Direct execution, but only after spawning an Explore subagent to map the utility's usage.

Scenario: Structured Data Extraction

Q13. Your extraction tool's JSON schema marks `vendor_tax_id` as required. Processing a batch of invoices, you find the model populates plausible-looking but fabricated tax IDs for the ~20% of invoices that don't print one. What is the correct fix?

A. Add a validation-retry loop that re-requests extraction when the tax ID fails a checksum. B. Add few-shot examples showing invoices without tax IDs. C. Make `vendor_tax_id` optional and nullable in the schema. D. Add a system prompt instruction: "Never fabricate values that are not present in the source document."

Q14. Your pipeline processes three document types with three different extraction schemas. The document type is not known until the model reads the document. Occasionally the model returns a conversational text response instead of calling any extraction tool. What configuration fixes this?

A. `tool_choice: {"type": "auto"}` B. `tool_choice: {"type": "any"}` C. `tool_choice: {"type": "tool", "name": "extract_invoice"}` D. `tool_choice: {"type": "none"}` with an output schema instead

---

Q15. Extraction validation fails on 6% of documents. Investigating, you find two distinct causes: (a) the model formats dates as `03/04/2026` when the schema expects ISO 8601, and (b) the requested purchase-order number appears only in a separate document not supplied to the model. You implement a retry loop that resends the document, the failed extraction, and the validation error. What outcome should you expect?

A. Both causes resolve, since the model can self-correct given specific validation errors. B. Cause (a) resolves; cause (b) does not, because the information is absent from the source. C. Neither resolves, because retries cannot fix schema-level failures. D. Cause (b) resolves; cause (a) does not, because format errors require a schema change.

Q16. Your team runs two Claude workloads: a nightly test-generation job whose output is reviewed the next morning, and a pre-commit hook that must return before the developer's commit completes. A cost review proposes moving both to the Message Batches API for the 50% savings. What is the correct assessment?

A. Move both; poll for completion and the batches typically finish in minutes. B. Move the nightly test generation only; keep the pre-commit hook on the synchronous API. C. Keep both synchronous; the Batches API cannot correlate responses to requests reliably. D. Move both, with a timeout fallback to the synchronous API if a batch exceeds five minutes.

Scenario: Claude Code for Continuous Integration

Q17. Your CI code review flags 40% false positives in the "unused variable" category and about 5% in the "SQL injection risk" category. Developers have started ignoring all automated comments, including the security ones. What should you do first?

A. Add "only report findings you are highly confident about" to the review prompt. B. Temporarily disable the unused-variable category while you improve its prompt, keeping the security category active. C. Require two independent review passes and post only findings that appear in both. D. Add a confidence score to each finding and post only those above 0.8.

Q18. Your CI job runs `claude "Review this diff for security issues" --output-format json` and the job never completes; logs show it waiting for input. What is the fix?

A. Add the `-p` flag. B. Set `CLAUDE_HEADLESS=true` in the job environment. C. Add `--json-schema` so the process knows to terminate after producing valid output. D. Add `--no-interactive`.

Q19. After a new commit, your CI re-runs the review on the updated PR. Developers complain that every previously-reported issue is posted again as a new comment, burying the new findings. What is the appropriate fix?

A. Deduplicate comments in the posting script by hashing the finding text. B. Include the prior review findings in context and instruct Claude to report only new or still-unaddressed issues. C. Only run the review on the incremental diff since the last review. D. Reduce review frequency to once per PR, at open time.

Scenario: Developer Productivity with Claude

Q20. Your agent has both the built-in `Grep` tool and an MCP-provided `search_code_semantically` tool that returns semantically ranked matches with surrounding context. In practice the agent almost always uses `Grep`, even for conceptual queries where semantic search would be far better. The MCP tool's description reads: "Searches code." What is the most effective first step?

A. Remove `Grep` from the agent's allowed tools so it must use the MCP tool. B. Expand the MCP tool's description to detail its capabilities, output format, example queries, and when to use it versus text search. C. Add a system prompt instruction: "CRITICAL: Always prefer `search_code_semantically` over `Grep`." D. Add 6-8 few-shot examples demonstrating conceptual queries routing to the MCP tool.

Scoring: record which domain each miss falls in. Any domain where you miss more than one third of its items goes on your day-10 focus list.

Domain 1 Drill - Agentic Architecture & Orchestration (15 questions)

The biggest domain: **27%** of the exam, about **16 of 60 questions**.

This drill is harder than Set 1 on purpose. Several questions have two answers that both look correct. The difference is usually one phrase in the question text.

The questions use exam-level English. The explanations are in simple English.

Time: 30 minutes. Answers at the bottom - cover them while you work.

Q1. Your agentic loop is implemented as:

```
for i in range(10):
   response = call_claude(messages)
   if "I have completed" in response_text:
       break
   execute_tools_and_append(response)
```

Production logs show two failure classes: the loop sometimes exits while the agent still has pending tool calls, and sometimes runs all 10 iterations on tasks that finished at iteration 3. Which change fixes both?

A. Raise the iteration cap to 25 and add more completion phrases to the text check. B. Branch on `stop_reason`: continue while it is `"tool_use"`, terminate when it is `"end_turn"`. C. Have the model call an explicit `task_complete` tool, and break when that tool is called. D. Check whether the response contains any text content blocks; if so, treat the turn as complete.

Q2. A coordinator agent spawns a synthesis subagent with the prompt: "Synthesize the research findings into a report." The synthesis subagent responds asking what findings it should synthesize. What is the cause?

A. The coordinator's `allowedTools` is missing `"Task"`. B. Subagents operate with isolated context and do not inherit the coordinator's conversation history; the findings must be included explicitly in the subagent's prompt. C. The synthesis subagent's AgentDefinition system prompt does not describe its role clearly enough. D. The coordinator needs to invoke the synthesis subagent with `fork_session` so it inherits the analysis baseline.

Q3. Your refund policy states that refunds over $500 require manager approval. You add to the system prompt: "IMPORTANT: You must never process a refund over $500. Always escalate these to a human." In a 5,000-conversation audit, 11 refunds over $500 were processed. What is the correct remediation?

A. Strengthen the instruction with explicit examples of over-threshold refunds being escalated. B. Move the threshold check into the `process_refund` tool's input validation, returning a business error. C. Implement a tool-call interception hook that blocks the call when the amount exceeds $500 and redirects to the escalation workflow. D. Lower the effective threshold in the prompt to $400 to create a safety margin.

Q4. Your research coordinator produces reports on "renewable energy adoption" that cover solar comprehensively and barely mention wind, hydro, or geothermal. Logs show three subtasks: "solar panel efficiency trends," "residential solar adoption rates," "solar policy incentives." Each subagent returned high-quality results. Which two changes address the root cause? **(Select two.)**

A. Design the coordinator to analyze query requirements and dynamically select subagents rather than always routing through a fixed pipeline. B. Partition research scope across subagents by assigning distinct subtopics, minimizing duplication and maximizing coverage. C. Implement an iterative refinement loop where the coordinator evaluates synthesis output for gaps and re-delegates with targeted queries. D. Instruct the synthesis subagent to flag topic areas that appear underrepresented in the findings it receives.

Q5. You need to compare two refactoring strategies for the same module, both starting from an expensive codebase analysis you've already completed in the current session. What mechanism is designed for this?

A. `--resume <session-name>` twice, once per strategy. B. `fork_session`, creating independent branches from the shared analysis baseline. C. Two Task tool calls emitted in a single response, one per strategy. D. `/compact`, then start each strategy as a fresh session with the compacted summary.

Q6. Your MCP tools return timestamps in three formats: `order_service` uses Unix epoch seconds, `billing_service` uses ISO 8601, and `legacy_crm` uses a numeric status code plus a separate date string. The agent frequently miscalculates date differences across services. What is the appropriate fix?

A. A `PostToolUse` hook that normalizes the heterogeneous formats before the model processes the results. B. A system prompt section documenting each service's timestamp format and conversion rules. C. Few-shot examples demonstrating correct date arithmetic for each format. D. A tool-call interception hook that rewrites requests to ask each service for ISO 8601.

Q7. You are automating "add comprehensive test coverage to a legacy billing module." You do not know the module's structure, which paths are highest-risk, or what the dependency graph looks like. Which decomposition strategy fits?

A. Prompt chaining: a fixed pipeline that analyzes each file, generates tests, then runs a coverage check. B. Dynamic adaptive decomposition: map the structure, identify high-impact areas, then build a prioritized plan that adapts as dependencies are discovered. C. Per-file local passes plus a separate cross-file integration pass. D. Spawn one subagent per file in parallel, each responsible for that file's tests.

Q8. A customer message reads: "My order #4471 arrived damaged, and separately I was charged twice for order #4390 last month." How should a well-designed agent handle this?

A. Address the damaged order first, then ask the customer to open a separate ticket for the billing issue. B. Decompose into two distinct items, investigate each in parallel using shared context, then synthesize a unified resolution. C. Escalate to a human, since multi-concern messages exceed the agent's single-issue design. D. Ask the customer which issue they would like resolved first.

Q9. You resume a named investigation session two days later with `--resume auth-refactor`. In the interim, three of the files the agent analyzed have been substantially rewritten by another developer. What should you do?

A. Resume as normal; the agent will re-read files as needed. B. Resume, and inform the agent about the specific file changes so it re-analyzes those targets rather than re-exploring everything. C. Start a fresh session, since any file change invalidates the prior analysis. D. Run `/compact` first to clear the stale file contents from context.

Q10. Your coordinator's system prompt reads: "Step 1: call the web search agent. Step 2: pass results to the document analyzer. Step 3: pass output to synthesis. Step 4: pass to report generation." Queries that need no document analysis still run the full pipeline, and complex queries that would benefit from a second search round never get one. What change addresses this?

A. Add conditional branches to the prompt describing when each step may be skipped. B. Write the coordinator prompt to specify research goals and quality criteria rather than step-by-step procedures, so it can dynamically select subagents and iterate. C. Add a routing classifier ahead of the coordinator that determines which pipeline stages to enable per query. D. Split into four separate coordinators, one per query type.

Q11. In a code review workflow, a single-pass review of a 22-file PR produces detailed findings for the first few files, superficial comments for the rest, and flags a pattern as a bug in one file while approving identical code in another. Which decomposition applies?

A. Dynamic adaptive decomposition based on what each file reveals. B. Prompt chaining: analyze each file individually for local issues, then run a separate cross-file integration pass. C. Run the review three times and report only findings that recur. D. Route each file to a separate subagent and have the coordinator merge results.

Q12. Your agent escalates a billing dispute to a human. The human agent replies in Slack: "I have no idea what this is about - what did the customer say, what did you check, and what do you want me to do?" What was missing?

A. A transcript link so the human can read the conversation. B. A structured handoff summary containing customer ID, root cause analysis, refund amount, and recommended action. C. A confidence score indicating how uncertain the agent was. D. The escalation should not have occurred; the agent should have resolved it autonomously.

Q13. Which of the following correctly describes the relationship between an agentic loop iteration and conversation history? **(Select two.)**

A. Tool results are appended to conversation history so the model can incorporate new information into its reasoning on the next iteration. B. Each `tool_use` block must receive a corresponding `tool_result` with the matching `tool_use_id`. C. Tool results should be summarized before appending, to prevent context growth. D. The assistant message containing `tool_use` blocks should be omitted from history once the results are known.

Q14. A coordinator delegates to four subagents. One subagent's tool fails with a transient network error, retries locally twice, and succeeds on the third attempt. What should it report to the coordinator?

A. The successful result. The transient failure was resolved locally and does not require coordinator involvement. B. The successful result plus a structured error record documenting the transient failures. C. A partial-failure status, so the coordinator can decide whether to trust the result. D. Nothing - subagents should propagate all errors upward for centralized handling.

Q15. Your synthesis subagent has been given the full tool set: `web_search`, `load_document`, `extract_data_points`, `verify_claim_against_source`, `fetch_url`, `summarize_content`, `generate_citation`, and 11 others - 18 tools total. It has begun performing its own web searches instead of synthesizing the findings it was given, and tool selection has become erratic. Which two changes address this? **(Select two.)**

A. Restrict the synthesis subagent's tool set to those relevant to its role. B. Provide a scoped `verify_fact` tool for the high-frequency simple verification case, routing complex verifications through the coordinator. C. Add a system prompt instruction forbidding the synthesis agent from performing web searches. D. Replace `fetch_url` with a constrained `load_document` tool that validates document URLs.

Domain 2 Drill - Tool Design & MCP Integration (15 questions)

18% of the exam, about 11 of 60 questions.

You scored 2 out of 2 in Set 1. That is only two questions, which is not enough to know if this domain is strong. This drill is harder. Several questions ask you to choose between two techniques that are **both real** and **both in the guide**.

The questions use exam-level English. The explanations are in simple English.

Time: 30 minutes. Answers below - cover them while you work.

Q1. Your research agent has a tool `analyze_document` described as: *"Analyzes a document and returns useful information."* It is used for three different jobs - pulling specific figures out of tables, producing an executive summary, and checking whether a claim is supported by a source. Output quality is erratic and callers can't predict what they'll get back. What is the most effective structural fix?

A. Expand the description to cover all three use cases with example queries for each. B. Split it into `extract_data_points`, `summarize_content`, and `verify_claim_against_source`, each with a defined input/output contract. C. Add an `analysis_type` enum parameter with three values and describe each in the parameter description. D. Add few-shot examples in the system prompt showing each of the three usage patterns.

Q2. Two tools exist: `analyze_content` (*"Analyzes content and extracts key information"*) and `analyze_document` (*"Analyzes documents and extracts key information"*). `analyze_content` is intended only for web search result pages. Misrouting is frequent. What is the appropriate fix?

A. Merge them into a single `analyze` tool that accepts a `source_type` parameter. B. Rename `analyze_content` to `extract_web_results` and rewrite its description to be web-specific. C. Add a system prompt rule: "Use `analyze_content` only for web results." D. Remove `analyze_content` and route web pages through `analyze_document`.

Q3. Tool descriptions in your system are detailed and clearly differentiated. Nonetheless the agent routes nearly every request mentioning the word "report" to `generate_report`, even when the user is asking to *read* an existing report. Your system prompt contains the line: *"Your primary purpose is to produce reports for stakeholders."* What should you investigate?

A. Whether `generate_report`'s description needs a stronger "when not to use" section. B. Whether the system prompt's keyword-sensitive wording is creating an unintended tool association that overrides the descriptions. C. Whether the agent has too many tools for reliable selection. D. Whether few-shot examples are needed for read-vs-generate disambiguation.

Q4. A customer requests a refund on an order that is 95 days old; your policy window is 90 days. The `process_refund` MCP tool must reject this. What should it return?

A. `{"isError": true, "message": "Operation failed"}` B. `{"isError": true, "errorCategory": "business", "retriable": false, "message": "Refunds are available within 90 days of purchase. This order was placed 95 days ago."}` C. `{"isError": true, "errorCategory": "validation", "isRetryable": true, "message": "Invalid refund request"}` D. A successful response with `refund_amount: 0`, letting the agent infer the rejection.

Q5. Your `search_knowledge_base` MCP tool needs to distinguish two outcomes: the search index was unreachable, and the search ran successfully but matched nothing. Currently both return an empty array. Why does this matter? **(Select two.)**

A. The agent cannot decide whether a retry is appropriate. B. The agent may tell the user no information exists when the index was simply down. C. Empty arrays consume unnecessary context tokens. D. The coordinator cannot annotate coverage gaps accurately in the final output.

Q6. A document-analysis subagent calls an MCP tool that fails with a rate-limit error. The subagent waits and retries; the second attempt succeeds. Separately, a different call fails because the requested document requires permissions the subagent does not have, and no retry will help. What should the subagent report to the coordinator?

A. Both failures, so the coordinator has complete visibility. B. Neither; the subagent should return only its final analysis. C. Only the permission failure, including what was attempted and any partial results. D. Only the rate-limit failure, since permission errors should be escalated to a human directly.

Q7. Your synthesis agent has 18 tools. Selection has become erratic and it sometimes performs its own web searches instead of synthesizing. Which statement best explains the mechanism?

A. Tool schemas consume context, leaving less room for the findings it must synthesize. B. Giving an agent access to too many tools degrades selection reliability by increasing decision complexity, and agents with tools outside their specialization tend to misuse them. C. The tool descriptions are individually adequate but collectively contradictory. D. The model's attention dilutes across tool definitions the same way it dilutes across files in a large review.

Q8. Evaluation shows your synthesis agent needs verification on 40% of its claims: 85% of those are simple fact-checks (dates, names, statistics) and 15% require multi-source investigation. Currently every verification round-trips through the coordinator to the web search agent. Which design best balances overhead against separation of concerns?

A. Give the synthesis agent the full web search tool set so it can verify anything directly. B. Give the synthesis agent a scoped `verify_fact` tool for simple lookups; route complex verifications through the coordinator as today. C. Have the synthesis agent batch all verification needs and submit them to the coordinator in one call at the end of its pass. D. Have the web search agent pre-cache extra context around every source so synthesis rarely needs to verify.

Q9. Your agent has a `fetch_url` tool. It has begun fetching arbitrary URLs it encounters in document text, including tracking links and unrelated pages. What is the appropriate change?

A. Add a system prompt instruction listing which URL patterns are permitted. B. Replace `fetch_url` with a constrained `load_document` tool that validates document URLs. C. Add a `PostToolUse` hook that discards responses from non-document URLs. D. Remove the fetch capability and require documents to be pre-loaded as resources.

Q10. Your extraction pipeline must always run `extract_metadata` before any enrichment tool, because enrichment depends on the document type that metadata determines. What configuration enforces this?

A. `tool_choice: {"type": "any"}` on the first request. B. `tool_choice: {"type": "tool", "name": "extract_metadata"}` on the first request, then process subsequent steps in follow-up turns. C. Order `extract_metadata` first in the `tools` array. D. A system prompt instruction stating that `extract_metadata` must always be called first.

Q11. Your team shares a Jira MCP server across the project. Individual developers also want a personal, experimental Notion server that shouldn't affect teammates. Where does each go?

A. Both in `.mcp.json`, with the Notion one commented out by default. B. Jira in `.mcp.json`; Notion in `~/.claude.json`. C. Jira in `~/.claude.json`; Notion in `.mcp.json`. D. Both in `~/.claude.json`, with the Jira config distributed via the team wiki.

Q12. Your `.mcp.json` needs a GitHub personal access token to authenticate its MCP server, and the file is committed to the repository. What is the correct approach?

A. Commit the token and rotate it monthly. B. Use environment variable expansion - `${GITHUB_TOKEN}` - in `.mcp.json`. C. Move the server config to `~/.claude.json` so the token isn't committed. D. Store the token in CLAUDE.md, which is excluded from version control.

Q13. Your agent repeatedly makes exploratory tool calls to discover what issues exist in your tracker, what documentation pages are available, and what tables the database has - burning turns before doing real work. Which MCP capability addresses this?

A. Increasing the tool count so each discovery has a dedicated tool. B. Exposing content catalogs (issue summaries, documentation hierarchies, database schemas) as MCP **resources**. C. Caching prior tool results in a scratchpad file the agent reads first. D. Enhancing each tool's description to explain what data it can return.

Q14. Your team needs Jira integration for a standard workflow - reading issues, transitioning status, adding comments. A well-maintained community MCP server exists. Your team also has a bespoke internal deployment-approval workflow with

no equivalent anywhere. What should you do?

A. Build custom servers for both, for consistency and control. B. Use the community server for Jira; build a custom server for the deployment-approval workflow. C. Use the community server for Jira and implement deployment approvals as built-in Bash tool calls. D. Fork the community Jira server and extend it to cover deployment approvals too.

Q15. You have an MCP tool `search_code_semantically` that returns ranked matches with context, and the built-in `Grep`. Match each task to the right tool. **(Select two correct pairings.)**

A. "Find every caller of `processPayment`" -> Grep B. "Find all files matching `**/*.integration.test.ts`" -> Grep C. "Find the code responsible for retry behavior in checkout" -> `search_code_semantically` D. "Find the exact string `ERR_TIMEOUT_4471`" -> `search_code_semantically`

Domain 3 Drill - Claude Code Configuration & Workflows (15 questions)

20% of the exam, about 12 of 60 questions.

You scored 4 out of 6 in Set 1. You missed Q9 (`@import` vs path rules) and Q12 (using plan mode when it was not needed). Both of those appear again here.

This is the easiest domain to improve. It is mostly memorisation: file paths, frontmatter keys, and command-line flags. Every mistake here is a fact you can learn in one day.

The questions use exam-level English. The explanations are in simple English.

Time: 30 minutes. Answers below - cover them while you work.

Q1. A new engineer joins and reports that Claude Code ignores the team's error-handling conventions, while everyone else's sessions apply them correctly. The conventions are documented and were working before she joined. Where should you look first?

A. Her local `.claude/settings.json` may be overriding the project configuration. B. The conventions are in a user-level `~/.claude/CLAUDE.md`, which is not shared via version control. C. Her Claude Code version predates `.claude/rules/` support. D. The project CLAUDE.md exceeds the size at which later sections stop being loaded.

Q2. Your monorepo has four packages. Each has a maintainer who should decide which of eight shared standards documents apply to their package. You want no duplication of those documents. What is the right mechanism?

A. `.claude/rules/` files with `paths:` globs, one per package directory. B. A CLAUDE.md in each package directory that uses `@import` to reference the applicable standards files. C. A root CLAUDE.md containing all eight documents under per-package headers. D. Eight skills in `.claude/skills/`, one per standards document, invoked as needed.

Q3. Your Terraform configuration lives under `terraform/` and has conventions that apply only when editing those files. You want them loaded automatically, and only then. What do you create?

A. `terraform/CLAUDE.md` with the conventions. B. A `.claude/rules/` file with YAML frontmatter `paths: ["terraform/**/*"]`. C. A skill in `.claude/skills/terraform/SKILL.md` with `argument-hint`. D. A section in the root CLAUDE.md headed "Terraform conventions."

Q4. Your test files sit beside the code they test - `Button.test.tsx` next to `Button.tsx`, `auth.test.ts` next to `auth.ts` - across roughly 60 directories. All tests must follow one set of conventions. Which approach works, and why do the others fail? Select the correct approach.

A. A `CLAUDE.md` in each of the 60 directories. B. A `.claude/rules/` file with `paths: ["**/*.test.*"]`. C. A root CLAUDE.md section titled "Testing conventions." D. A skill containing the testing conventions, invoked before writing tests.

Q5. Claude Code behaves inconsistently across sessions in the same repo - sometimes applying your commit conventions, sometimes not. What is the first diagnostic step?

A. `/compact` B. `/memory` C. `claude --resume` D. Delete and recreate `.claude/`

Q6. You've written a `/security-review` command your whole team should get automatically on `git pull`. Where does the file go?

A. `~/.claude/commands/security-review.md` B. `.claude/commands/security-review.md` C. `.claude/skills/security-review/SKILL.md` D. A `commands` array in `.claude/config.json`

Q7. Your team has a shared `/deploy-check` skill. You want a personal variant that also runs your own extra linters, without changing your teammates' behavior. What do you do?

A. Edit `.claude/skills/deploy-check/SKILL.md` and add your linters behind a conditional. B. Create a variant in `~/.claude/skills/` **with a different name**. C. Create `~/.claude/skills/deploy-check/SKILL.md`, overriding the project skill by name. D. Add your linters to `~/.claude/CLAUDE.md`.

Q8. Match each SKILL.md frontmatter option to the problem it solves. **(Select three correct pairings.)**

A. Skill produces thousands of lines of analysis that displace the main task -> `context: fork` B. Skill could delete files if it misinterprets an instruction -> `allowed-tools` C. Developers invoke the skill without the parameter it needs -> `argument-hint` D. Skill should load automatically when editing matching files -> `paths:`

Q9. You need to decide between a skill and CLAUDE.md for two pieces of content: (i) your team's universal naming and error-handling standards, and (ii) a multi-step database migration procedure used a few times a quarter. Which goes where?

A. Both in CLAUDE.md; skills are for tool restrictions only. B. (i) CLAUDE.md - always-loaded universal standards; (ii) a skill - on-demand, task-specific workflow. C. (i) a skill - so it can restrict tools; (ii) CLAUDE.md - so it's always available when needed. D. Both as skills, invoked by a rule in `.claude/rules/`.

Q10. You must migrate a library used in 45+ files, and there are two viable migration strategies with different infrastructure implications. Which approach?

A. Direct execution with a detailed upfront specification of the target state. B. Plan mode to explore the codebase and design the approach, then direct execution to implement the chosen plan. C. Direct execution, switching to plan mode if unexpected complexity appears. D. Plan mode throughout, including implementation.

Q11. A stack trace identifies a missing null check on line 88 of `dateUtils.ts`. The fix is one conditional. Which approach?

A. Plan mode, because `dateUtils` is imported in many places. B. Direct execution. C. Plan mode, then direct execution. D. Spawn an Explore subagent to map usage, then direct execution.

Q12. During a multi-phase refactor, the discovery phase produces enormous output - file listings, dependency traces, search results - and by phase three the main agent is giving inconsistent answers about what it found in phase one. Which two mechanisms address this? **(Select two.)**

A. Use the Explore subagent for the verbose discovery phases, returning summaries. B. Maintain a scratchpad file of key findings and reference it in later phases. C. Increase the session's context window setting. D. Restart the session between each phase with no carryover.

Q13. Your CI job runs:

```
claude "Review this PR for security issues" --output-format json
```

The job hangs indefinitely; logs show it waiting for interactive input. Which single change fixes it?

A. Add `--json-schema` so it terminates once valid output is produced. B. Add `-p`. C. Set `CLAUDE_HEADLESS=true`. D. Redirect stdin from `/dev/null`.

Q14. Your CI review posts findings as inline PR comments via a script that parses Claude Code's output. The parser breaks whenever the prose format shifts. Which two flags make the output reliably machine-parseable? **(Select two.)**

A. `--output-format json` B. `--json-schema` C. `--print` D. `--strict`

Q15. Your CI review runs again after each new commit. Developers complain that resolved and unresolved issues alike are re-posted every run, burying new findings. Separately, your test-generation job keeps proposing tests for scenarios already covered. What addresses both? **(Select two.)**

A. Include prior review findings in context and instruct Claude to report only new or still-unaddressed issues. B. Provide the existing test files in context so generation avoids duplicate scenarios. C. Document testing standards and available fixtures in CLAUDE.md. D. Run both jobs only once, at PR open time.

Domain 4 Drill - Prompt Engineering & Structured Output (15 questions)

20% of the exam, about 12 of 60 questions.

This is your weakest domain. You scored 1 out of 5 in Set 1. All three misses were flat facts: `tool_choice` values, required schema fields, and which errors a retry can fix. Those three facts are tested again here, in different clothes.

Four questions ask for **two answers**. The real exam is about one in five, so this matches.

The questions use exam-level English. The explanations are in simple English.

Time: 30 minutes. Answers below - cover them while you work.

Scenario: Structured Data Extraction

Q1. Your extraction schema marks `contract_end_date` as required. Reviewing 400 processed contracts, you find that for the ~15% that are open-ended agreements with no end date, the model has populated the field with dates that appear nowhere in the source document - most commonly one year after the start date. Which change stops this?

A. Add a validation rule rejecting any `contract_end_date` exactly 365 days after `contract_start_date`. B. Make `contract_end_date` optional and nullable in the schema. C. Add a system prompt instruction: "Do not infer or calculate dates that are not explicitly stated in the document." D. Add few-shot examples showing open-ended contracts extracted with the date field empty.

Q2. Your pipeline handles three document types - invoices, receipts and purchase orders - each with its own extraction tool and schema. The type is not known until the document is read. You observe that roughly a quarter of requests return a paragraph describing the document rather than calling any tool. What is the correct configuration?

A. `tool_choice: {"type": "auto"}` with a stronger system prompt instruction to always extract. B. `tool_choice: {"type": "any"}` C. `tool_choice: {"type": "tool", "name": "extract_invoice"}`, since invoices are the most common type. D. Merge the three tools into one with a `document_type` enum parameter.

Q3. Your enrichment pipeline must call `extract_metadata` before `lookup_vendor_details`, because the vendor lookup depends on the country code that metadata extraction produces. In production, the model occasionally calls them in the wrong order. Which approach enforces the ordering?

A. `tool_choice: {"type": "any"}` on the first request. B. `tool_choice: {"type": "tool", "name": "extract_metadata"}` on the first request, then handle the enrichment steps in follow-up turns. C. List `extract_metadata` before `lookup_vendor_details` in the `tools` array. D. Describe the dependency in both tool descriptions.

Q4. After switching from free-text JSON output to tool use with a strict JSON schema, your malformed-JSON error rate drops from 3% to zero. However, your finance team reports that invoice line items still sometimes fail to sum to the stated total, and occasionally the tax amount appears in the shipping field. Which two statements are correct? **(Select two.)**

A. Strict schemas eliminate syntax errors but do not prevent semantic errors. B. The remaining failures indicate the schema was not marked `strict: true` correctly. C. Extracting `calculated_total` alongside `stated_total` would let you detect the summing problem. D. Switching back to free-text JSON with a stronger prompt would fix the field placement.

Q5. Validation fails on 8% of your extractions. Investigating a sample, you identify two causes: some documents express amounts as "1.2M" where the schema expects a number, and some documents genuinely do not contain the requested vendor registration number - it exists only in a separate supplier master file you do not pass to the model. You implement a retry loop that resends the document, the failed extraction, and the specific validation error. What should you expect?

A. Both causes resolve, because the model can self-correct when given the specific error. B. The "1.2M" formatting cases resolve; the missing registration numbers do not. C. Neither resolves, because retries cannot correct schema-level failures. D. The missing registration numbers resolve; the formatting cases require a schema change.

Q6. Your extraction handles supplier documents from twelve countries. The `payment_terms` field currently uses a free-text string, and downstream systems cannot process the variety. You want a constrained field that still handles terms you have not anticipated. What schema design achieves this?

A. A required string field with format normalisation rules in the prompt. B. An enum of the known payment terms, plus an `"other"` value and a separate detail string field. C. An enum of the known payment terms, with validation rejecting anything unrecognised. D. A nullable enum, so unrecognised terms produce `null`.

Q7. You are processing a batch of 100 scanned documents through the Message Batches API. Eighteen fail. Investigating, you find twelve exceeded the context limit and six hit a transient service error. Which two actions are correct? **(Select two.)**

A. Resubmit only the eighteen failed documents, identified by their `custom_id` values. B. Resubmit the entire batch of 100, since partial resubmission risks inconsistent processing. C. Chunk the twelve oversized documents before resubmitting them. D. Match the results to requests by their position in the response array.

Q8. Your nightly extraction job processes about 8,000 documents. Documents arrive continuously throughout the day, and your commitment to the downstream team is that any document is processed within 30 hours of arrival. You use the Message Batches API. What submission cadence meets the commitment?

A. Submit once daily at midnight. B. Submit every 12 hours. C. Submit every 4 hours. D. Submit each document individually as it arrives.

Q9. Before running a 10,000-document batch, what is the most cost-effective preparation step?

A. Increase `max_tokens` so no extraction is truncated. B. Refine the prompt against a small sample set to maximise the first-pass success rate. C. Split the batch into ten batches of 1,000 to reduce the impact of any single failure. D. Run the same batch twice and compare, to identify non-deterministic extractions.

Scenario: Claude Code for Continuous Integration

Q10. Your automated review flags 34% false positives in its "unnecessary abstraction" category. Your team lead suggests adding "only report issues you are highly confident are genuine problems" to the review prompt. What is wrong with this suggestion, and what should you do instead?

A. Nothing is wrong; confidence-based instructions are the documented approach for precision. B. General confidence instructions do not improve precision. Define specific categorical criteria for what to report and what to skip. C. The instruction is correct but must be paired with a numeric confidence threshold in the output schema. D. The instruction is correct but should be placed in CLAUDE.md rather than the review prompt.

Q11. Your review produces severity ratings that are inconsistent between runs - the same class of issue is rated "high" in one review and "low" in another. What produces consistent classification?

A. Instructing the model to be consistent with previous reviews of the same repository. B. Defining explicit severity criteria with concrete code examples for each severity level. C. Reducing the severity scale from five levels to two. D. Having a second review pass re-rate the severities assigned by the first.

Q12. Your review comments are technically accurate but developers say they are hard to act on - the format varies between findings, and some omit the file location or the suggested fix. Detailed written instructions about the required format have not fixed it. Which two changes address this? **(Select two.)**

A. Add few-shot examples demonstrating the exact output shape: location, issue, severity, suggested fix. B. Enforce the output shape with a JSON schema via tool use. C. Add "IMPORTANT: always include the file location" to the top of the prompt. D. Increase `max_tokens` so findings are not truncated mid-format.

Q13. Your team runs two Claude workloads. The first is a pre-merge review that blocks the merge button until it returns. The second is a weekly architecture-drift report, read at Monday's planning meeting. Finance asks you to cut costs. What do you do?

A. Move both to the Message Batches API for the 50% saving. B. Move the weekly report to the Batches API; keep the pre-merge review synchronous. C. Keep both synchronous, because batch results cannot be reliably matched to requests. D. Move both to the Batches API, with a fallback to the synchronous API if a batch takes longer than 10 minutes.

Q14. Your CI generates code and then, in the same session, reviews that generated code before opening the pull request. Reviewers report the automated review rarely finds problems that later turn out to be real. What is the underlying cause, and what fixes it?

A. The review prompt lacks explicit criteria; add categorical criteria for what to report. B. The model retains its reasoning context from generation and does not question its own decisions. Use a separate, independent instance to review. C. The review runs before tests, so it lacks failure information. Reorder the pipeline. D. Extended thinking is not enabled on the review step. Enable it so the model reasons more carefully.

Q15. A pull request touches 18 files in the payments module. Your single review pass produces detailed findings for the first few files, thin findings for the rest, and in two cases flags a pattern as a defect in one file while approving the identical pattern in another. Which two changes address this? **(Select two.)**

A. Run a separate review pass per file, for issues local to that file. B. Run an additional pass across all files, examining data flow between them. C. Move to a model with a larger context window so all 18 files fit comfortably. D. Run the review three times and report only findings that appear in at least two runs.
